Deep Interest Network for Click-Through Rate Prediction

Guorui Zhou, Chengru Song, Xiaoqiang Zhu
Ying Fan, Han Zhu, Xiao Ma, Yanghui Yan, Junqi Jin, Han Li, Kun Gai
Alibaba Group
{guorui.xgr,chengru.scr,xiaoqiang.zxq,zhuhan.zh,fanying.fy,maxiao.ma,yanghui.yyh,junqi.jjq,lihan.hl,jingshi.gk}@alibaba-inc.com

arXiv:1706.06978v4 [stat.ML] 13 Sep 2018

ABSTRACT

Click-through rate prediction is an essential task in industrial applications, such as online advertising. Recently deep learning based models have been proposed, which follow a similar Embedding&MLP paradigm. In these methods large scale sparse input features are first mapped into low dimensional embedding vectors, and then transformed into fixed-length vectors in a group-wise manner, finally concatenated together to fed into a multilayer perceptron (MLP) to learn the nonlinear relations among features. In this way, user features are compressed into a fixed-length representation vector, in regardless of what candidate ads are. The use of fixed-length vector will be a bottleneck, which brings difficulty for Embedding&MLP methods to capture user's diverse interests effectively from rich historical behaviors. In this paper, we propose a novel model: Deep Interest Network (DIN) which tackles this challenge by designing a local activation unit to adaptively learn the representation of user interests from historical behaviors with respect to a certain ad. This representation vector varies over different ads, improving the expressive ability of model greatly. Besides, we develop two techniques: mini-batch aware regularization and data adaptive activation function which can help training industrial deep networks with hundreds of millions of parameters. Experiments on two public datasets as well as an Alibaba real production dataset with over 2 billion samples demonstrate the effectiveness of proposed approaches, which achieve superior performance compared with state-of-the-art methods. DIN now has been successfully deployed in the online display advertising system in Alibaba, serving the main traffic.

CCS CONCEPTS

• **Information systems** → **Display advertising**; **Recommender systems**;

KEYWORDS

Click-Through Rate Prediction, Display Advertising, E-commerce

1 INTRODUCTION

In cost-per-click (CPC) advertising system, advertisements are ranked by the eCPM (effective cost per mille), which is the product of the bid price and CTR (click-through rate), and CTR needs to be predicted by the system. Hence, the performance of CTR prediction model has a direct impact on the final revenue and plays a key role in the advertising system. Modeling CTR prediction has received much attention from both research and industry community.

Recently, inspired by the success of deep learning in computer vision [14] and natural language processing [1], deep learning based methods have been proposed for CTR prediction task [3, 4, 21, 26]. These methods follow a similar Embedding&MLP paradigm: large scale sparse input features are first mapped into low dimensional embedding vectors, and then transformed into fixed-length vectors in a group-wise manner, finally concatenated together to fed into fully connected layers (also known as multilayer perceptron, MLP) to learn the nonlinear relations among features. Compared with commonly used logistic regression model [19], these deep learning methods can reduce a lot of feature engineering jobs and enhance the model capability greatly. For simplicity, we name these methods Embedding&MLP in this paper, which now have become popular on CTR prediction task.

However, the user representation vector with a limited dimension in Embedding&MLP methods will be a bottleneck to express user's diverse interests. Take display advertising in e-commerce site as an example. Users might be interested in different kinds of goods simultaneously when visiting the e-commerce site. That is to say, user interests are ***diverse***. When it comes to CTR prediction task, user interests are usually captured from user behavior data. Embedding&MLP methods learn the representation of all interests for a certain user by transforming the embedding vectors of user behaviors into a fixed-length vector, which is in an euclidean space where all users' representation vectors are. In other words, diverse interests of the user are compressed into a fixed-length vector, which limits the expressive ability of Embedding&MLP methods. To make the representation capable enough for expressing user's diverse interests, the dimension of the fixed-length vector needs to be largely expanded. Unfortunately, it will dramatically enlarge the size of learning parameters and aggravate the risk of overfitting under limited data. Besides, it adds the burden of computation and storage, which may not be tolerated for an industrial online system.

On the other hand, it is not necessary to compress all the diverse interests of a certain user into the same vector when predicting a candidate ad because only part of user's interests will influence his/her action (to click or not to click). For example, a female swimmer will click a recommended goggle mostly due to the bought of bathing suit rather than the shoes in her last week's shopping list. Motivated by this, we propose a novel model: Deep Interest Network (DIN), which adaptively calculates the representation vector of user interests by taking into consideration the relevance of historical behaviors given a candidate ad. By introducing a local activation unit, DIN pays attentions to the related user interests by soft-searching for relevant parts of historical behaviors and takes a weighted sum pooling to obtain the representation of user interests with respect to the candidate ad. Behaviors with higher relevance to the candidate ad get higher activated weights and dominate the representation of user interests. We visualize this phenomenon in the experiment section. In this way, the representation vector of

user interests varies over different ads, which improves the expressive ability of model under limited dimension and enables DIN to better capture user's diverse interests.

Training industrial deep networks with large scale sparse features is of great challenge. For example, SGD based optimization methods only update those parameters of sparse features appearing in each mini-batch. However, adding with traditional ℓ_2 regularization, the computation turns to be unacceptable, which needs to calculate L2-norm over the whole parameters (with size scaling up to billions in our situation) for each mini-batch. In this paper, we develop a novel mini-batch aware regularization where only parameters of non-zero features appearing in each mini-batch participate in the calculation of L2-norm, making the computation acceptable. Besides, we design a data adaptive activation function, which generalizes commonly used PReLU[12] by adaptively adjusting the rectified point w.r.t. distribution of inputs and is shown to be helpful for training industrial networks with sparse features.

The contributions of this paper are summarized as follows:

- We point out the limit of using fixed-length vector to express user's diverse interests and design a novel deep interest network (DIN) which introduces a local activation unit to adaptively learn the representation of user interests from historical behaviors w.r.t. given ads. DIN can improve the expressive ability of model greatly and better capture the diversity characteristic of user interests.
- We develop two novel techniques to help training industrial deep networks: i) a mini-batch aware regularizer, which saves heavy computation of regularization on deep networks with huge number of parameters and is helpful for avoiding overfitting, ii) a data adaptive activation function, which generalizes PReLU by considering the distribution of inputs and shows well performance.
- We conduct extensive experiments on both public and Alibaba datasets. Results verify the effectiveness of proposed DIN and training techniques. Our code[1] is publicly available. The proposed approaches have been deployed in the commercial display advertising system in Alibaba, one of world's largest advertising platform, contributing significant improvement to the business.

In this paper we focus on the CTR prediction modeling in the scenario of display advertising in e-commerce industry. Methods discussed here can be applied in similar scenarios with rich user behaviors, such as personalized recommendation in e-commerce sites, feeds ranking in social networks etc.

The rest of the paper is organized as follows. We discuss related work in section 2 and introduce the background about characteristic of user behavior data in display advertising system of e-commerce site in section 3. Section 4 and 5 describe in detail the design of DIN model as well as two proposed training techniques. We present experiments in section 6 and conclude in section 7.

2 RELATEDWORK

The structure of CTR prediction model has evolved from shallow to deep. At the same time, the number of samples and the dimension of features used in CTR model have become larger and larger. In order to better extract feature relations to improve performance, several works pay attention to the design of model structure.

As a pioneer work, NNLM [2] learns distributed representation for each word, aiming to avoid curse of dimension in language modeling. This method, often referred to as embedding, has inspired many natural language models and CTR prediction models that need to handle large-scale sparse inputs.

LS-PLM [9] and FM [20] models can be viewed as a class of networks with one hidden layer, which first employs embedding layer on sparse inputs and then imposes specially designed transformation functions for target fitting, aiming to capture the combination relations among features.

Deep Crossing [21], Wide&Deep Learning [4] and YouTube Recommendation CTR model [3] extend LS-PLM and FM by replacing the transformation function with complex MLP network, which enhances the model capability greatly. PNN[5] tries to capture high-order feature interactions by involving a product layer after embedding layer. DeepFM[10] imposes a factorization machines as "wide" module in Wide&Deep [4] with no need of feature engineering. Overall, these methods follow a similar model structure with combination of embedding layer (for learning the dense representation of sparse features) and MLP (for learning the combination relations of features automatically). This kind of CTR prediction model reduces the manual feature engineering jobs greatly. Our base model follows this kind of model structure. However in applications with rich user behaviors, features are often contained with variable-length list of ids, e.g., searched terms or watched videos in YouTube recommender system [3]. These models often transform corresponding list of embedding vectors into a fixed-length vector via sum/average pooling, which causes loss of information. The proposed DIN tackles it by adaptively learning the representation vector w.r.t. given ad, improving the expressive ability of model.

Attention mechanism originates from Neural Machine Translation (NMT) field [1]. NMT takes a weighted sum of all the annotations to get an expected annotation and focuses only on information relevant to the generation of next target word. A recent work, DeepIntent [26] applies attention in the context of search advertising. Similar to NMT, they use RNN[24] to model text, then learn one global hidden vector to help paying attention on the key words in each query. It is shown that the use of attention can help capturing the main intent of query or ad. DIN designs a local activation unit to soft-search for relevant user behaviors and takes a weighted sum pooling to obtain the adaptive representation of user interests with respect to a given ad. The user representation vector varies over different ads, which is different from DeepIntent in which there is no interaction between ad and user.

We make code publicly available, and further show how to successfully deploy DIN in one of the world's largest advertising systems with novel developed techniques for training large scale deep networks with hundreds of millions of parameters.

3 BACKGROUND

In e-commerce sites, such as Alibaba, advertisements are natural goods. In the rest of this paper, without special declaration, we regard ads as goods. Figure 1 briefly illustrates the running procedure

[1]Experiment code on two public datasets is available on GitHub: https://github.com/zhougr1993/DeepInterestNetwork

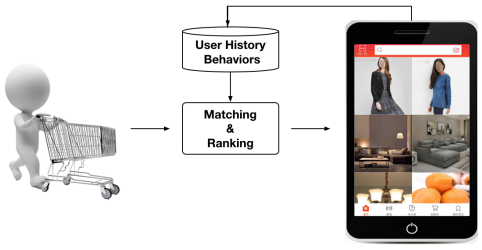


Figure 1: Illustration of running procedure of display advertising system in Alibaba, in which user behavior data plays important roles.

of display advertising system in Alibaba, which consists of two main stages: i) matching stage which generates list of candidate ads relevant to the visiting user via methods like collaborative filtering, ii) ranking stage which predicts CTR for each given ad and then selects top ranked ones. Everyday, hundreds of millions of users visit the e-commerce site, leaving us with lots of user behavior data which contributes critically in building matching and ranking models. It is worth mentioning that users with rich historical behaviors contain diverse interests. For example, a young mother has browsed goods including woolen coat, T-shits, earrings, tote bag, leather handbag and children's coat recently. These behavior data give us hints about her shopping interests. When she visits the e-commerce site, system displays a suitable ad to her, for example a new handbag. Obviously the displayed ad only matches or activates part of interests of this mother. In summary, interests of user with rich behaviors are ***diverse*** and could be ***locally activated*** given certain ads. We show later in this paper making use of these characteristics plays important role for building CTR prediction model.

4 DEEP INTEREST NETWORK

Different from sponsored search, users come into display advertising system without explicitly expressed intentions. Effective approaches are required to extract user interests from rich historical behaviors when building the CTR prediction model. Features that depict users and ads are the basic elements in the CTR modeling of advertisement system. Making use of these features reasonably and mining information from them are critical.

4.1 Feature Representation

Data in industrial CTR prediction tasks is mostly in a multi-group categorial form, for example, [weekday=Friday, gender=Female, visited_cate_ids={Bag,Book}, ad_cate_id=Book], which is normally transformed into high-dimensional sparse binary features via encoding [4, 19, 21]. Mathematically, encoding vector of i-th feature group is formularized as $\mathbf{t}_i \in R^{K_i}$. K_i denotes the dimensionality of feature group i, which means feature group i contains K_i unique ids. $\mathbf{t}_i[j]$ is the j-th element of $\mathbf{t}_i$ and $\mathbf{t}_i[j] \in \{0, 1\}$. $\sum_{j=1}^{K_i} \mathbf{t}_i[j] = k$. Vector $\mathbf{t}_i$ with $k = 1$ refers to one-hot encoding and $k > 1$ refers to multi-hot encoding. Then one instance can be represent as $\boldsymbol{x} = [\boldsymbol{t}_1^T, \boldsymbol{t}_2^T, ...\boldsymbol{t}_M^T]^T$ in a group-wise manner, where M is number of feature groups, $\sum_{i=1}^{M} K_i = K$, K is dimensionality of the entire feature space. In this way, the aforementioned instance with

Table 1: Statistics of feature sets used in the display advertising system in Alibaba. Features are composed of sparse binary vectors in the group-wise manner.

Category	Feature Group	Dimemsionality	Type	#Nonzero Ids per Instance
User Profile Features	gender	2	one-hot	1
	age_level	~ 10	one-hot	1
	...	...	...	...
User Behavior Features	visited_goods_ids	~ 10^9	multi-hot	~ 10^3
	visited_shop_ids	~ 10^7	multi-hot	~ 10^3
	visited_cate_ids	~ 10^4	multi-hot	~ 10^2
Ad Features	goods_id	~ 10^7	one-hot	1
	shop_id	~ 10^5	one-hot	1
	cate_id	~ 10^4	one-hot	1
	...	...	...	...
Context Features	pid	~ 10	one-hot	1
	time	~ 10	one-hot	1
	...	...	...	...

four groups of features are illustrated as:

$$\underbrace{[0, 0, 0, 0, 1, 0, 0]}_{\text{weekday=Friday}} \quad \underbrace{[0, 1]}_{\text{gender=Female}} \quad \underbrace{[0, .., 1, ..., 1, ...0]}_{\text{visited_cate_ids=\{Bag,Book\}}} \quad \underbrace{[0, .., 1, ..., 0]}_{\text{ad_cate_id=Book}}$$

The whole feature set used in our system is described in Table 1. It is composed of four categories, among which user behavior features are typically multi-hot encoding vectors and contain rich information of user interests. Note that in our setting, there are no combination features. We capture the interaction of features with deep neural network.

4.2 Base Model(Embedding&MLP)

Most of the popular model structures [3, 4, 21] share a similar Embedding&MLP paradigm, which we refer to as base model, as shown in the left of Fig.2. It consists of several parts:

Embedding layer. As the inputs are high dimensional binary vectors, embedding layer is used to transform them into low dimensional dense representations. For the i-th feature group of $\boldsymbol{t}_i$, let $\mathrm{W}^i = [w_1^i, ..., w_j^i, ..., w_{K_i}^i] \in \mathbb{R}^{D \times K_i}$ represent the i-th embedding dictionary, where $w_j^i \in R^D$ is an embedding vector with dimensionality of D. Embedding operation follows the table lookup mechanism, as illustrated in Fig.2.

- If $\boldsymbol{t}_i$ is one-hot vector with j-th element $\boldsymbol{t}_i[j] = 1$, the embedded representation of $\boldsymbol{t}_i$ is a single embedding vector $\boldsymbol{e}_i = w_j^i$.
- If $\boldsymbol{t}_i$ is multi-hot vector with $\boldsymbol{t}_i[j] = 1$ for $j \in \{i_1, i_2, ..., i_k\}$, the embedded representation of $\boldsymbol{t}_i$ is a list of embedding vectors: $\{\boldsymbol{e}_{i_1}, \boldsymbol{e}_{i_2}, ...\boldsymbol{e}_{i_k}\} = \{w_{i_1}^i, w_{i_2}^i, ...w_{i_k}^i\}$.

Pooling layer and Concat layer. Notice that different users have different numbers of behaviors. Thus the number of non-zero values for multi-hot behavioral feature vector $\boldsymbol{t}_i$ varies across instances, causing the lengths of the corresponding list of embedding vectors to be variable. As fully connected networks can only handle fixed-length inputs, it is a common practice [3, 4] to transform the list of embedding vectors via a pooling layer to get a fixed-length vector:

$$\boldsymbol{e}_i = \text{pooling}(\boldsymbol{e}_{i_1}, \boldsymbol{e}_{i_2}, ...\boldsymbol{e}_{i_k}). \tag{1}$$

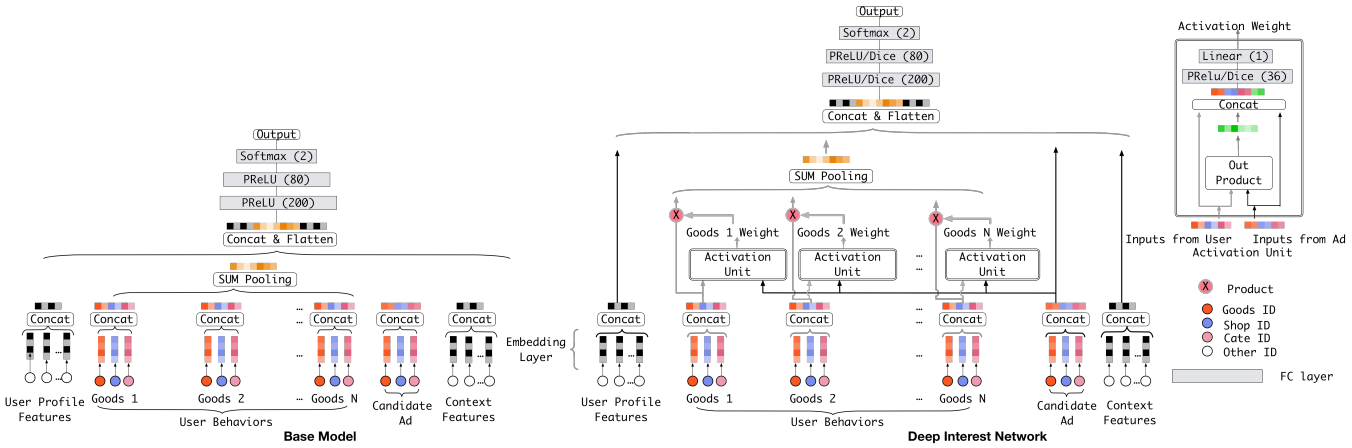


Figure 2: Network Architecture. The left part illustrates the network of base model (Embedding&MLP). Embeddings of cate_id, shop_id and goods_id belong to one goods are concatenated to represent one visited goods in user's behaviors. Right part is our proposed DIN model. It introduces a local activation unit, with which the representation of user interests varies adaptively given different candidate ads.

Two most commonly used pooling layers are sum pooling and average pooling, which apply element-wise sum/average operations to the list of embedding vectors.

Both embedding and pooling layers operate in a group-wise manner, mapping the original sparse features into multiple fixed-length representation vectors. Then all the vectors are concatenated together to obtain the overall representation vector for the instance.

MLP. Given the concatenated dense representation vector, fully connected layers are used to learn the combination of features automatically. Recently developed methods [4, 5, 10] focus on designing structures of MLP for better information extraction.

Loss. The objective function used in base model is the negative log-likelihood function defined as:

$$L = -\frac{1}{N}\sum_{(\boldsymbol{x},y)\in\mathcal{S}} (y\log p(\boldsymbol{x}) + (1-y)\log(1-p(\boldsymbol{x}))), \tag{2}$$

where $\mathcal{S}$ is the training set of size N, with $\boldsymbol{x}$ as the input of the network and $y \in \{0,1\}$ as the label, $p(\boldsymbol{x})$ is the output of the network after the softmax layer, representing the predicted probability of sample $\boldsymbol{x}$ being clicked.

4.3 The structure of Deep Interest Network

Among all those features of Table 1, user behavior features are critically important and play key roles in modeling user interests in the scenario of e-commerce applications.

Base model obtains a fixed-length representation vector of user interests by pooling all the embedding vectors over the user behavior feature group, as Eq.(1). This representation vector stays the same for a given user, in regardless of what candidate ads are. In this way, the user representation vector with a limited dimension will be a bottleneck to express user's diverse interests. To make it capable enough, an easy method is to expand the dimension of embedding vector, which unfortunately will increase the size of learning parameters heavily. It will lead to overfitting under limited training data and add the burden of computation and storage, which may not be tolerated for an industrial online system.

Is there an elegant way to represent user's diverse interests in one vector under limited dimension? The local activation characteristic of user interests gives us inspiration to design a novel model named deep interest network(DIN). Imagine when the young mother mentioned above in section 3 visits the e-commerce site, she finds the displayed new handbag cute and clicks it. Let's dissect the driving force of click action. The displayed ad hits the related interests of this young mother by soft-searching her historical behaviors and finding that she had browsed similar goods of tote bag and leather handbag recently. In other words, behaviors related to displayed ad greatly contribute to the click action. DIN simulates this process by paying attention to the representation of locally activated interests w.r.t. given ad. Instead of expressing all user's diverse interests with the same vector, DIN adaptively calculate the representation vector of user interests by taking into consideration the relevance of historical behaviors w.r.t. candidate ad. This representation vector varies over different ads.

The right part of Fig.2 illustrates the architecture of DIN. Compared with base model, DIN introduces a novel designed local activation unit and maintains the other structures the same. Specifically, activation units are applied on the user behavior features, which performs as a weighted sum pooling to adaptively calculate user representation $\boldsymbol{v}_U$ given a candidate ad A, as shown in Eq.(3)

$$\boldsymbol{v}_U(A) = f(\boldsymbol{v}_A, \boldsymbol{e}_1, \boldsymbol{e}_2, .., \boldsymbol{e}_H) = \sum_{j=1}^{H} a(\boldsymbol{e}_j, \boldsymbol{v}_A)\boldsymbol{e}_j = \sum_{j=1}^{H} \boldsymbol{w}_j \boldsymbol{e}_j, \tag{3}$$

where $\{\boldsymbol{e}_1, \boldsymbol{e}_2, ..., \boldsymbol{e}_H\}$ is the list of embedding vectors of behaviors of user U with length of H, $\boldsymbol{v}_A$ is the embedding vector of ad A. In this way, $\boldsymbol{v}_U(A)$ varies over different ads. $a(\cdot)$ is a feed-forward network with output as the activation weight, as illustrated in Fig.2. Apart from the two input embedding vectors, $a(\cdot)$ adds the out

product of them to feed into the subsequent network, which is an explicit knowledge to help relevance modeling.

Local activation unit of Eq.(3) shares similar ideas with attention methods which are developed in NMT task[1]. However different from traditional attention method, the constraint of $\sum_i w_i = 1$ is relaxed in Eq.(3), aiming to reserve the intensity of user interests. That is, normalization with softmax on the output of $a(\cdot)$ is abandoned. Instead, value of $\sum_i w_i$ is treated as an approximation of the intensity of activated user interests to some degree. For example, if one user's historical behaviors contain 90% clothes and 10% electronics. Given two candidate ads of T-shirt and phone, T-shirt activates most of the historical behaviors belonging to clothes and may get larger value of $\boldsymbol{v}_U$ (higher intensity of interest) than phone. Traditional attention methods lose the resolution on the numerical scale of $\boldsymbol{v}_U$ by normalizing of the output of $a(\cdot)$.

We have tried LSTM to model user historical behavior data in the sequential manner. But it shows no improvement. Different from text which is under the constraint of grammar in NLP task, the sequence of user historical behaviors may contain multiple concurrent interests. Rapid jumping and sudden ending over these interests causes the sequence data of user behaviors to seem to be noisy. A possible direction is to design special structures to model such data in a sequence way. We leave it for future research.

5 TRAINING TECHNIQUES

In the advertising system in Alibaba, numbers of goods and users scale up to hundreds of millions. Practically, training industrial deep networks with large scale sparse input features is of great challenge. In this section, we introduce two important techniques which are proven to be helpful in practice.

5.1 Mini-batch Aware Regularization

Overfitting is a critical challenge for training industrial networks. For example, with addition of fine-grained features, such as features of *goods_ids* with dimensionality of 0.6 billion (including *visited_goods_ids* of user and *goods_id* of ad as described in Table 1), model performance falls rapidly after the first epoch during training without regularization, as the dark green line shown in Fig.4 in later section 6.5. It is not practical to directly apply traditional regularization methods, such as ℓ_2 and ℓ_1 regularization, on training networks with sparse inputs and hundreds of millions of parameters. Take ℓ_2 regularization as an example. Only parameters of non-zero sparse features appearing in each mini-batch needs to be updated in the scenario of SGD based optimization methods without regularization. However, when adding ℓ_2 regularization it needs to calculate L2-norm over the whole parameters for each mini-batch, which leads to extremely heavy computations and is unacceptable with parameters scaling up to hundreds of millions.

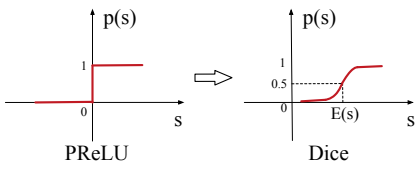


Figure 3: Control function of PReLU and Dice.

In this paper, we introduce an efficient mini-batch aware regularizer, which only calculates the L2-norm over the parameters of sparse features appearing in each mini-batch and makes the computation possible. In fact, it is the embedding dictionary that contributes most of the parameters for CTR networks and arises the difficulty of heavy computation. Let $\mathbf{W} \in \mathbb{R}^{D\times K}$ denote parameters of the whole embedding dictionary, with D as the dimensionality of the embedding vector and K as the dimensionality of feature space. Expand the ℓ_2 regularization on $\mathbf{W}$ over samples

$$L_2(\mathbf{W}) = \|\mathbf{W}\|_2^2 = \sum_{j=1}^{K} \|\boldsymbol{w}_j\|_2^2 = \sum_{(\boldsymbol{x},y)\in\mathcal{S}} \sum_{j=1}^{K} \frac{I(\boldsymbol{x}_j \neq 0)}{n_j} \|\boldsymbol{w}_j\|_2^2, \tag{4}$$

where $\boldsymbol{w}_j \in \mathbb{R}^D$ is the j-th embedding vector, $I(\boldsymbol{x}_j \neq 0)$ denotes if the instance $\boldsymbol{x}$ has the feature id j, and n_j denotes the number of occurrence for feature id j in all samples. Eq.(4) can be transformed into Eq.(5) in the mini-batch aware manner

$$L_2(\mathbf{W}) = \sum_{j=1}^{K} \sum_{m=1}^{B} \sum_{(\boldsymbol{x},y)\in\mathcal{B}_m} \frac{I(\boldsymbol{x}_j \neq 0)}{n_j} \|\boldsymbol{w}_j\|_2^2, \tag{5}$$

where B denotes the number of mini-batches, $\mathcal{B}_m$ denotes the m-th mini-batch. Let $\alpha_{mj} = \max_{(\boldsymbol{x},y)\in\mathcal{B}_m} I(\boldsymbol{x}_j \neq 0)$ denote if there is at least one instance having the feature id j in mini-batch $\mathcal{B}_m$. Then Eq.(5) can be approximated by

$$L_2(\mathbf{W}) \approx \sum_{j=1}^{K} \sum_{m=1}^{B} \frac{\alpha_{mj}}{n_j} \|\boldsymbol{w}_j\|_2^2. \tag{6}$$

In this way, we derive an approximated mini-batch aware version of ℓ_2 regularization. For the m-th mini-batch, the gradient w.r.t. the embedding weights of feature j is

$$\boldsymbol{w}_j \leftarrow \boldsymbol{w}_j - \eta \left[\frac{1}{|\mathcal{B}_m|} \sum_{(\boldsymbol{x},y)\in\mathcal{B}_m} \frac{\partial L(p(\boldsymbol{x}), y)}{\partial \boldsymbol{w}_j} + \lambda \frac{\alpha_{mj}}{n_j} \boldsymbol{w}_j \right], \tag{7}$$

in which only parameters of features appearing in m-th mini-batch participate in the computation of regularization.

5.2 Data Adaptive Activation Function

PReLU [12] is a commonly used activation function

$$f(s) = \begin{cases} s & \text{if } s > 0 \\ \alpha s & \text{if } s \leq 0. \end{cases} = p(s) \cdot s + (1 - p(s)) \cdot \alpha s, \tag{8}$$

where s is one dimension of the input of activation function $f(\cdot)$ and $p(s) = I(s > 0)$ is an indicator function which controls $f(s)$ to switch between two channels of $f(s) = s$ and $f(s) = \alpha s$. α in the second channel is a learning parameter. Here we refer to $p(s)$ as the control function. The left part of Fig.3 plots the control function of PReLU. PReLU takes a hard rectified point with value of 0, which may be not suitable when the inputs of each layer follow different distributions. Take this into consideration, we design a novel data adaptive activation function named ***Dice***,

$$f(s) = p(s) \cdot s + (1 - p(s)) \cdot \alpha s, \quad p(s) = \frac{1}{1 + e^{-\frac{s - E[s]}{\sqrt{Var[s] + \epsilon}}}} \tag{9}$$

with the control function to be plotted in the right part of Fig.3. In the training phrase, $E[s]$ and $Var[s]$ is the mean and variance of input in each mini-batch. In the testing phrase, $E[s]$ and $Var[s]$ is calculated by moving averages $E[s]$ and $Var[s]$ over data. ϵ is a small constant which is set to be 10^{-8} in our practice.

Dice can be viewed as a generalization of PReLu. The key idea of Dice is to adaptively adjust the rectified point according to distribution of input data, whose value is set to be the mean of input. Besides, Dice controls smoothly to switch between the two channels. When $E(s) = 0$ and $Var[s] = 0$, Dice degenerates into PReLU.

6 EXPERIMENTS

In this section, we present our experiments in detail, including datasets, evaluation metric, experimental setup, model comparison and the corresponding analysis. Experiments on two public datasets with user behaviors as well as a dataset collected from the display advertising system in Alibaba demonstrate the effectiveness of proposed approach which outperforms state-of-the-art methods on the CTR prediction task. Both the public datasets and experiment codes are made available[1].

6.1 Datasets and Experimental Setup

Amazon Dataset[2]. Amazon Dataset contains product reviews and metadata from Amazon, which is used as benchmark dataset[13, 18, 23]. We conduct experiments on a subset named Electronics, which contains 192,403 users, 63,001 goods, 801 categories and 1,689,188 samples. User behaviors in this dataset are rich, with more than 5 reviews for each users and goods. Features include goods_id, cate_id, user reviewed goods_id_list and cate_id_list. Let all behaviors of a user be $(b_1, b_2, \ldots, b_k, \ldots, b_n)$, the task is to predict the (k+1)-th reviewed goods by making use of the first k reviewed goods. Training dataset is generated with $k = 1, 2, \ldots, n-2$ for each user. In the test set, we predict the last one given the first $n-1$ reviewed goods. For all models, we use SGD as the optimizer with exponential decay, in which learning rate starts at 1 and decay rate is set to 0.1. The mini-batch size is set to be 32.

MovieLens Dataset[3]. MovieLens data[11] contains 138,493 users, 27,278 movies, 21 categories and 20,000,263 samples. To make it suitable for CTR prediction task, we transform it into a binary classification data. Original user rating of the movies is continuous value ranging from 0 to 5. We label the samples with rating of 4 and 5 to be positive and the rest to be negative. We segment the data into training and testing dataset based on userID. Among all 138,493 users, of which 100,000 are randomly selected into training set (about 14,470,000 samples) and the rest 38,493 into the test set (about 5,530,000 samples). The task is to predict whether user will rate a given movie to be above 3(positive label) based on historical behaviors. Features include movie_id, movie_cate_id and user rated movie_id_list, movie_cate_id_list. We use the same optimizer, learning rate and mini-batch size as described on Amazon Dataset.

Alibaba Dataset. We collected traffic logs from the online display advertising system in Alibaba, of which two weeks' samples are used for training and samples of the following day for testing. The size of training and testing set is about 2 billions and 0.14 billion respectively. For all the deep models, the dimensionality of embedding vector is 12 for the whole 16 groups of features. Layers of MLP is set by $192 \times 200 \times 80 \times 2$. Due to the huge size of data, we set the mini-batch size to be 5000 and use Adam[15] as the optimizer. We apply exponential decay, in which learning rate starts at 0.001 and decay rate is set to 0.9.

[2]http://jmcauley.ucsd.edu/data/amazon/

[3]https://grouplens.org/datasets/movielens/20m/

Table 2: Statistics of datasets used in this paper.

Dataset	Users	Goods[a]	Categories	Samples
Amazon(Electro).	192,403	63,001	801	1,689,188
MovieLens.	138,493	27,278	21	20,000,263
Alibaba.	60 million	0.6 billion	100,000	2.14 billion

[a] For MovieLens dataset, goods refer to be movies.

The statistics of all the above datasets is shown in Table 2. Volume of Alibaba Dataset is much larger than both Amazon and MovieLens, which brings more challenges.

6.2 Competitors

- **LR[19]**. Logistic regression (LR) is a widely used shallow model before deep networks for CTR prediction task. We implement it as a weak baseline.
- **BaseModel**. As introduced in section4.2, BaseModel follows the Embedding&MLP architecture and is the base of most of subsequently developed deep networks for CTR modeling. It acts as a strong baseline for our model comparison.
- **Wide&Deep[4]**. In real industrial applications, Wide&Deep model has been widely accepted. It consists of two parts: i) wide model, which handles the manually designed cross product features, ii) deep model, which automatically extracts nonlinear relations among features and equals to the BaseModel. Wide&Deep needs expertise feature engineering on the input of the "wide" module. We follow the practice in [10] to take cross-product of user behaviors and candidates as wide inputs. For example, in MovieLens dataset, it refers to the cross-product of user rated movies and candidate movies.
- **PNN[5].** PNN can be viewed as an improved version of BaseModel by introducing a product layer after embedding layer to capture high-order feature interactions.
- **DeepFM[10]**. It imposes a factorization machines as "wide" module in Wide&Deep saving feature engineering jobs.

6.3 Metrics

In CTR prediction field, AUC is a widely used metric[8]. It measures the goodness of order by ranking all the ads with predicted CTR, including intra-user and inter-user orders. An variation of user weighted AUC is introduced in [7, 13] which measures the goodness of intra-user order by averaging AUC over users and is shown to be more relevant to online performance in display advertising system. We adapt this metric in our experiments. For simplicity, we still refer it as AUC. It is calculated as follows:

$$\text{AUC} = \frac{\sum_{i=1}^{n} \#impression_i \times \text{AUC}_i}{\sum_{i=1}^{n} \#impression_i}, \tag{10}$$

where n is the number of users, $\#impression_i$ and AUC_i are the number of impressions and AUC corresponding to the i-th user.

Besides, we follow [25] to introduce RelaImpr metric to measure relative improvement over models. For a random guesser, the value of AUC is 0.5. Hence RelaImpr is defined as below:

$$RelaImpr = \left(\frac{\text{AUC(measured model)} - 0.5}{\text{AUC(base model)} - 0.5} - 1 \right) \times 100\%. \tag{11}$$

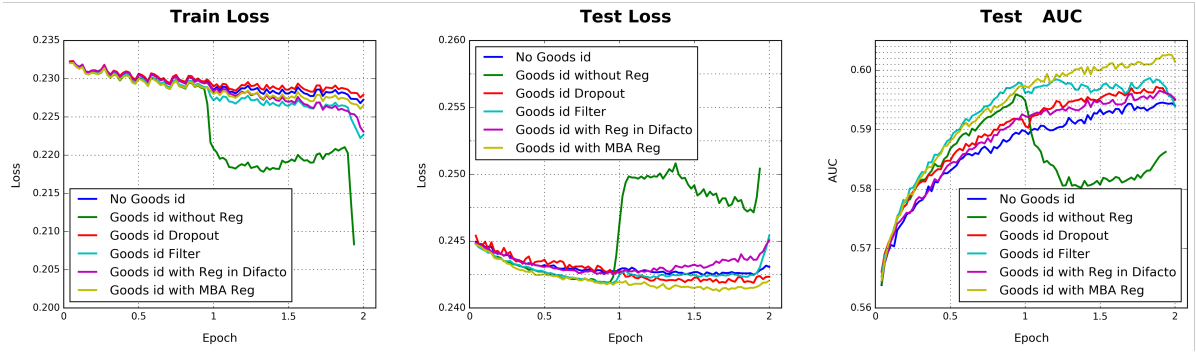


Figure 4: Performances of BaseModel with different regularizations on Alibaba Dataset. Training with fine-grained *goods_ids* features without regularization encounters serious overfitting after the first epoch. All the regularizations show improvement, among which our proposed mini-batch aware regularization performs best. Besides, well trained model with *goods_ids* features gets higher AUC than without them. It comes from the richer information that fine-grained features contained.

Table 3: Model Coparison on Amazon Dataset and MovieLens Dataset. All the lines calculate RelaImpr by comparing with BaseModel on each dataset respectively.

Model	MovieLens.		Amazon(Electro).	
	AUC	RelaImpr	AUC	RelaImpr
LR	0.7263	-1.61%	0.7742	-24.34%
BaseModel	0.7300	0.00%	0.8624	0.00%
Wide&Deep	0.7304	0.17%	0.8637	0.36%
PNN	0.7321	0.91%	0.8679	1.52%
DeepFM	0.7324	1.04%	0.8683	1.63%
DIN	**0.7337**	**1.61%**	**0.8818**	**5.35%**
DIN with Dice[a]	**0.7348**	**2.09%**	**0.8871**	**6.82%**

[a] Other lines except LR use PReLU as activation function.

6.4 Result from model comparison on Amazon Dataset and MovieLens Dataset

Table 3 shows the results on Amazon dataset and MovieLens dataset. All experiments are repeated 5 times and averaged results are reported. The influence of random initialization on AUC is less than 0.0002. Obviously, all the deep networks beat LR model significantly, which indeed demonstrates the power of deep learning. PNN and DeepFM with specially designed structures preform better than Wide&Deep. DIN performs best among all the competitors. Especially on Amazon Dataset with rich user behaviors, DIN stands out significantly. We owe this to the design of local activation unit structure in DIN. DIN pays attentions to the locally related user interests by soft-searching for parts of user behaviors that are relevant to candidate ad. With this mechanism, DIN obtains an adaptively varying representation of user interests, greatly improving the expressive ability of model compared with other deep networks. Besides, DIN with Dice brings further improvement over DIN, which verifies the effectiveness of the proposed data adaptive activation function Dice.

Table 4: Best AUCs of BaseModel with different regularizations on Alibaba Dataset corresponding to Fig.4. All the other lines calculate RelaImpr by comparing with first line.

Regularization	AUC	RelaImpr
Without goods_ids feature and Reg.	0.5940	0.00%
With goods_ids feature without Reg.	0.5959	2.02%
With goods_ids feature and Dropout Reg.	0.5970	3.19%
With goods_ids feature and Filter Reg.	0.5983	4.57%
With goods_ids feature and Difacto Reg.	0.5954	1.49%
With goods_ids feature and MBA. Reg.	**0.6031**	**9.68%**

6.5 Performance of regularization

As the dimension of features in both Amazon Dataset and MovieLens Dataset is not high (about 0.1 million), all the deep models including our proposed DIN do not meet grave problem of overfitting. However, when it comes to the Alibaba dataset from the online advertising system which contains higher dimensional sparse features, overfitting turns to be a big challenge. For example, when training deep models with fine-grained features (e.g., features of *goods_ids* with dimension of 0.6 billion in Table 1), serious overfitting occurs after the first epoch without any regularization, which causes the model performance to drop rapidly, as the dark green line shown in Fig.4. For this reason, we conduct careful experiments to check the performance of several commonly used regularizations.

- **Dropout[22]**. Randomly discard 50% of feature ids in each sample.
- **Filter**. Filter visited *goods_id* by occurrence frequency in samples and leave only the most frequent ones. In our setting, top 20 million *goods_ids* are left.
- **Regularization in DiFacto[16]**. Parameters associated with frequent features are less over-regularized.
- **MBA**. Our proposed **M**ini-**B**atch **A**ware regularization method (Eq.4). Regularization parameter λ for both DiFacto and MBA is searched and set to be 0.01.

Fig.4 and Table 4 give the comparison results. Focusing on the detail of Fig.4, model trained with fine-grained *goods_ids* features

brings large improvement on the test AUC performance in the first epoch, compared without it. However, overfitting occurs rapidly in the case of training without regularization (dark green line). Dropout prevents quick overfitting but causes slower convergence. Frequency filter relieves overfitting to a degree. Regularization in DiFacto sets a greater penalty on *goods_id* with high frequency, which performs worse than frequency filter. Our proposed mini-batch aware(MBA) regularization performs best compared with all the other methods, which prevents overfitting significantly.

Besides, well trained models with *goods_ids* features show better AUC performance than without them. This is duo to the richer information that fine-grained features contained. Considering this, although frequency filter performs slightly better than dropout, it throws away most of low frequent ids and may lose room for models to make better use of fine-grained features.

6.6 Result from model comparison on Alibaba Dataset

Table 5 shows the experimental results on Alibaba dataset with full feature sets as shown in Table 1. As expected, LR is proven to be much weaker than deep models. Making comparisons among deep models, we report several conclusions. First, under the same activation function and regularization, DIN itself has achieved superior performance compared with all the other deep networks including BaseModel, Wide&Deep, PNN and DeepFM. DIN achieves 0.0059 absolute AUC gain and 6.08% RelaImpr over BaseModel. It validates again the useful design of local activation unit structure. Second, ablation study based on DIN demonstrates the effectiveness of our proposed training techniques. Training DIN with mini-batch aware regularizer brings additional 0.0031 absolute AUC gain over dropout. Besides, DIN with Dice brings additional 0.0015 absolute AUC gain over PReLU.

Taken together, DIN with MBA regularization and Dice achieves total 11.65% RelaImpr and 0.0113 absolute AUC gain over BaseModel. Even compared with competitor DeepFM which performs best on this dataset, DIN still achieves 0.009 absolute AUC gain. It is notable that in commercial advertising systems with hundreds of millions of traffics, 0.001 absolute AUC gain is significant and worthy of model deployment empirically. DIN shows great superiority to better understand and make use of the characteristics of user behavior data. Besides, the two proposed techniques further improve model performance and provide powerful help for training large scale industrial deep networks.

6.7 Result from online A/B testing

Careful online A/B testing in the display advertising system in Alibaba was conducted from 2017-05 to 2017-06. During almost a month's testing, DIN trained with the proposed regularizer and activation function contributes up to 10.0% CTR and 3.8% RPM(Revenue Per Mille) promotion[4] compared with the introduced BaseModel, the last version of our online-serving model. This is a significant improvement and demonstrates the effectiveness of our proposed approaches. Now DIN has been deployed online and serves the main traffic.

[4]In our real advertising system, ads are ranked by $CTR^{\alpha} \cdot bid\text{-}price$ with $\alpha > 1.0$, which controls the balance of promotion of CTR and RPM.

Table 5: Model Comparison on Alibaba Dataset with full feature sets. All the lines calculate RelaImpr by comparing with BaseModel. DIN significantly outperforms all the other competitors. Besides, training DIN with our proposed mini-batch aware regularizer and Dice activation function brings further improvements.

Model	AUC	RelaImpr
LR	0.5738	- 23.92%
BaseModel[a,b]	0.5970	0.00%
Wide&Deep[a,b]	0.5977	0.72%
PNN[a,b]	0.5983	1.34%
DeepFM[a,b]	0.5993	2.37%
DIN Model[a,b]	**0.6029**	**6.08%**
DIN with MBA Reg.[a]	**0.6060**	**9.28%**
DIN with Dice [b]	**0.6044**	**7.63%**
DIN with MBA Reg. and Dice	**0.6083**	**11.65%**

[a] These lines are trained with PReLU as the activation function.
[b] These lines are trained with dropout regularization.

It is worth mentioning that online serving of industrial deep networks is not an easy job with hundreds of millions of users visiting our system everyday. Even worse, at traffic peak our system serves more than 1 million users per second. It is required to make realtime CTR predictions with high throughput and low latency. For example, in our real system we need to predict hundreds of ads for each visitor in less than 10 milliseconds. In our practice, several important techniques are deployed for accelerating online serving of industrial deep networks under the CPU-GPU architecture: i) *request batching* which merges adjacent requests from CPU to take advantage of GPU power, ii) *GPU memory optimization* which improves the access pattern to reduce wasted transactions in GPU memory, iii) *concurrent kernel computation* which allows execution of matrix computations to be processed with multiple CUDA kernels concurrently. In all, optimization of these techniques doubles the QPS (Query Per Second) capacity of a single machine practically. Online serving of DIN also benefits from this.

6.8 Visualization of DIN

Finally we conduct case study to reveal the inner structure of DIN on Alibaba dataset. We first examine the effectiveness of local activation unit. Fig.5 illustrates the activation intensity of user behaviors with respect to a candidate ad. As expected, behaviors with high relevance to candidate ad are weighted high.

We then visualize the learned embedding vectors. Taking the young mother mentioned before as example, we randomly select 9 categories (dress, sport shoes, bags, etc) and 100 goods of each category as the candidate ads for her. Fig.6 shows the visualization of embedding vectors of goods with t-SNE[17] learned by DIN, in which points with same shape correspond to the same category. We can see that goods with same category almost belong to one cluster, which shows the clustering property of DIN embeddings clearly. Besides, we color the points that represent candidate ads by the prediction value. Fig.6 is also a heat map of this mother's interest density distribution for potential candidates in embedding space. It shows DIN can form a multimodal interest density distribution in



Figure 5: Illustration of adaptive activation in DIN. Behaviors with high relevance to candidate ad get high activation weight.

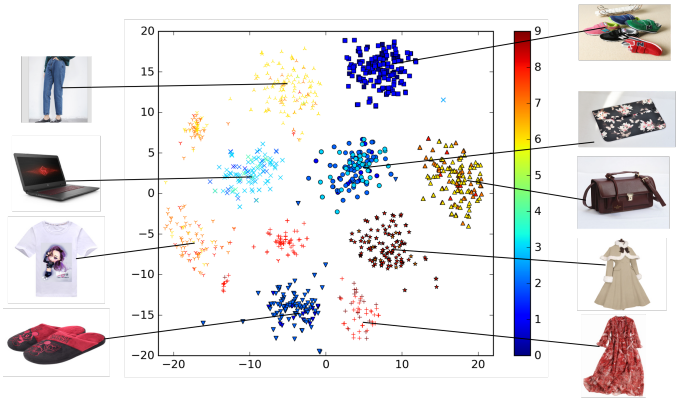


Figure 6: Visualization of embeddings of goods in DIN. Shape of points represents category of goods. Color of points corresponds to CTR prediction value.

candidates' embedding space for a certain user to capture his/her diverse interests.

7 CONCLUSIONS

In this paper, we focus on the task of CTR prediction modeling in the scenario of display advertising in e-commerce industry with rich user behavior data. The use of fixed-length representation in traditional deep CTR models is a bottleneck for capturing the diversity of user interests. To improve the expressive ability of model, a novel approach named DIN is designed to activate related user behaviors and obtain an adaptive representation vector for user interests which varies over different ads. Besides two novel techniques are introduced to help training industrial deep networks and further improve the performance of DIN. They can be easily generalized to other industrial deep learning tasks. DIN now has been deployed in the online display advertising system in Alibaba.

REFERENCES

[1] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. 2015. Neural Machine Translation by Jointly Learning to Align and Translate. In *Proceedings of the 3rd International Conference on Learning Representations*.

[2] Ducharme Réjean Bengio Yoshua et al. 2003. A neural probabilistic language model. *Journal of Machine Learning Research* (2003), 1137–1155.

[3] Paul Covington, Jay Adams, and Emre Sargin. 2016. Deep neural networks for youtube recommendations. In *Proceedings of the 10th ACM Conference on Recommender Systems*. ACM, 191–198.

[4] Cheng H. et al. 2016. Wide & deep learning for recommender systems. In *Proceedings of the 1st Workshop on Deep Learning for Recommender Systems*. ACM.

[5] Qu Y. et al. 2016. Product-Based Neural Networks for User Response Prediction. In *Proceedings of the 16th International Conference on Data Mining*.

[6] Wang H. et al. 2018. DKN: Deep Knowledge-Aware Network for News Recommendation. In *Proceedings of 26th International World Wide Web Conference*.

[7] Zhu H. et al. 2017. Optimized Cost per Click in Taobao Display Advertising. In *Proceedings of the 23rd International Conference on Knowledge Discovery and Data Mining*. ACM, 2191–2200.

[8] Tom Fawcett. 2006. An introduction to ROC analysis. *Pattern recognition letters* 27, 8 (2006), 861–874.

[9] Kun Gai, Xiaoqiang Zhu, et al. 2017. Learning Piece-wise Linear Models from Large Scale Data for Ad Click Prediction. *arXiv preprint arXiv:1704.05194* (2017).

[10] Huifeng Guo, Ruiming Tang, et al. 2017. DeepFM: A Factorization-Machine based Neural Network for CTR Prediction. In *Proceedings of the 26th International Joint Conference on Artificial Intelligence*. 1725–1731.

[11] F. Maxwell Harper and Joseph A. Konstan. 2015. The MovieLens Datasets: History and Context. *ACM Transactions on Interactive Intelligent Systems* 5, 4 (2015).

[12] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2015. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the IEEE International Conference on Computer Vision*. 1026–1034.

[13] Ruining He and Julian McAuley. 2016. Ups and Downs: Modeling the Visual Evolution of Fashion Trends with One-Class Collaborative Filtering. In *Proceedings of the 25th International Conference on World Wide Web*. 507–517. https://doi.org/10.1145/2872427.2883037

[14] Gao Huang, Zhuang Liu, Laurens van der Maaten, and Kilian Q. Weinberger. Densely connected convolutional networks.

[15] Diederik Kingma and Jimmy Ba. 2015. Adam: A Method for Stochastic Optimization. In *Proceedings of the 3rd International Conference on Learning Representations*.

[16] Mu Li, Ziqi Liu, Alexander J Smola, and Yu-Xiang Wang. 2016. DiFacto: Distributed factorization machines. In *Proceedings of the 9th ACM International Conference on Web Search and Data Mining*. 377–386.

[17] Laurens van der Maaten and Geoffrey Hinton. 2008. Visualizing data using t-SNE. *Journal of Machine Learning Research* 9, Nov (2008), 2579–2605.

[18] Julian Mcauley, Christopher Targett, Qinfeng Shi, and Van Den Hengel Anton. Image-Based Recommendations on Styles and Substitutes. In *Proceedings of the 38th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 43–52.

[19] H. Brendan Mcmahan, H. Brendan Holt, et al. 2014. Ad Click Prediction: a View from the Trenches. In *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 1222–1230.

[20] Steffen Rendle. 2010. Factorization machines. In *Proceedings of the 10th International Conference on Data Mining*. IEEE, 995–1000.

[21] Ying Shan, T Ryan Hoens, Jian Jiao, Haijing Wang, Dong Yu, and JC Mao. Deep Crossing: Web-scale modeling without manually crafted combinatorial features.

[22] Nitish Srivastava, Geoffrey E Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. 2014. Dropout: a simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research* 15, 1 (2014), 1929–1958.

[23] Andreas Veit, Balazs Kovacs, et al. 2015. Learning Visual Clothing Style With Heterogeneous Dyadic Co-Occurrences. In *Proceedings of the IEEE International Conference on Computer Vision*.

[24] Ronald J Williams and David Zipser. 1989. A learning algorithm for continually running fully recurrent neural networks. *Neural computation* (1989), 270–280.

[25] Ling Yan, Wu-jun Li, Gui-Rong Xue, and Dingyi Han. 2014. Coupled group lasso for web-scale ctr prediction in display advertising. In *Proceedings of the 31th International Conference on Machine Learning*. 802–810.

[26] Shuangfei Zhai, Keng-hao Chang, Ruofei Zhang, and Zhongfei Mark Zhang. 2016. Deepintent: Learning attentions for online advertising with recurrent neural networks. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 1295–1304.

[27] Song J et al. Zhou C, Bai J. 2018. ATRank: An Attention-Based User Behavior Modeling Framework for Recommendation. In *Proceedings of 32th AAAI Conference on Artificial Intelligence*.